SDS - REQ-IF-002 감사로그 기록 인터페이스

작성일 2026-10-17 · 근거: 요구사항정의서_엑셀양식_v3_1_3.xlsx (F열) · SRS 없음 · 대상: D:₩legacy-portal

1. 개요

요구사항 ID	REQ-IF-002
요구사항명	감사로그 기록 인터페이스
대 / 중 / 소분류	PORTAL / Common / Audit
우선순위	Medium
진행현황	테스트중 (비고: 소스 역추출.)
요약	결재·공지·일정의 상태 변경을 `AuditLogger` 인터페이스로 기록한다. 줄 형식 `[AUDIT] [yyyy-MM-dd HH:mm:ss] {동작} id={대상} by={수행자}`(+ 공백 + 상세)은 `ConsoleAuditLogger` 가 조립하고, 업무 코드는 동작 이름·대상·수행자만 넘긴다. 상태 변경이 실제로 일어났을 때만 기록한다.

2. 근거

SRS 없음 (엑셀 F열 · 소스만으로 분석). 아래는 요구사항정의서 F열(상세내용) 원문이다.

```
[개요]
결재 · 공지 · 일정의 상태 변경을 AuditLogger 인터페이스로 한 형식의 감사로그로 남긴다.

[상세 기능 요구사항]
1) AuditLogger.write(동작, 대상 ID, 수행자 ID[, 상세]) 로 기록한다.
2) 형식: '[AUDIT] [yyyy-MM-dd HH:mm:ss] {동작} id={대상} by={수행자}' 뒤에 상세가 있으면 공백 하나와 함께 붙인다.
3) 동작 이름: APPROVAL CREATE/SUBMIT/APPROVE/REJECT/CANCEL, NOTICE CREATE/PUBLISH, SCHEDULE CREATE/CONFIRM.
4) 상태 변경이 실제로 일어났을 때만 기록한다.

[입력 / 출력]
- 입력: 동작 이름, 대상 ID, 수행자 ID, 상세(선택)
- 출력: 감사로그 1줄(현재 콘솔 출력)

[제약 조건 / 비기능 요구사항]
- 로그 형식 · 동작 이름을 바꾸지 않는다.
- 형식 조립은 ConsoleAuditLogger 한 곳에만 둔다.

[검증 기준]
- 조건 불만족으로 무시된 처리에는 감사로그가 없다.
- 결재 생성 로그 끝에 'type={유형}' 이 붙는다.

[작업 대상]
legacy-portal (D:\legacy-portal) Spring Boot 3.2 / Java 21
src\main\java\kr\hull\portal\common (AuditLogger · ConsoleAuditLogger)
src\main\java\kr\hull\portal\approval\ApprovalAuditRecorder.java

[Worker Settings]
- MainWorker          : claude-main
```

```
- SubWorker           : codex-critic
- MainWorker-SubAgent : backend-expert
```

3. 분석 결과

3.1 추적표

항목	요구사항(F열 문장)	SRS 근거	소스 위치	판정	비고
상세 1	AuditLogger.write(동작, 대상 ID, 수행자 ID[, 상세]) 로 기록한다.	SRS 없음	`common/AuditLogger.java:15-25`	**충족**	오버로드 2개: `write(String action, Long id, Long userId)`, `write(String action, Long id, Long userId, String detail)`.
상세 2	형식: '[AUDIT] [yyyy-MM-dd HH:mm:ss] {동작} id={대상} by={수행자}' 뒤에 상세가 있으면 공백 하나와 함께 붙인다.	SRS 없음	`common/ConsoleAuditLogger.java:30, 44-61`	**충족**	시각 패턴 상수(:30), `"[" + now + "] " + action + " id=" + id + " by=" + userId`(:56), 상세는 `+ " " + detail`(:51), 접두 `"[AUDIT] "`(:60). 시각은 주입 Clock 기준(:55).
상세 3	동작 이름: APPROVAL CREATE/SUBMIT/APPROVE/REJECT/CANCEL, NOTICE CREATE/PUBLISH, SCHEDULE CREATE/CONFIRM.	SRS 없음	`approval/ApprovalAuditRecorder.java:27, 31, 35, 39, 43`, `notice/NoticeService.java:69, 94`, `schedule/ScheduleService.java:54, 91`	**충족**	9개 이름 모두 확인, 그 외 동작 이름 없음.
상세 4	상태 변경이 실제로 일어났을 때만 기록한다.	SRS 없음	`approval/ApprovalService.java:80-84, 140-172`, `notice/NoticeService.java:66-70, 87-94`, `schedule/ScheduleService.java:46-54, 87-91`	**충족**	처리 메서드마다 엔티티 행위가 false 면 return 후 기록 코드에 도달하지 않음. 생성(CREATE)은 저장 성공 시 항상 기록, 일정 생성은 검증·겹침 통과 후에만.
입력	입력: 동작 이름, 대상 ID, 수행자 ID, 상세(선택)	SRS 없음	`common/AuditLogger.java:18, 24`	**충족**	
출력	출력: 감사로그 1줄(현재 콘솔 출력)	SRS 없음	`common/ConsoleAuditLogger.java:59-61`	**충족**	`System.out.println` 1회.
제약 1	로그 형식 · 동작 이름을 바꾸지 않는다.	SRS 없음	`common/ConsoleAuditLogger.java:54-61`, 위 동작 이름 위치	**충족**	특성화 테스트 3종(결재·공지·일정)과 `ClockInjectionTest` 가 문자열을 고정.

항목	요구사항(F열 문장)	SRS 근거	소스 위치	판정	비고
제약 2	형식 조립은 ConsoleAuditLogger 한 곳에만 둔다.	SRS 없음	`common/ConsoleAuditLogger.java:54-61`, `common/FileAuditLogger.java:7-12`	**부분 충족**	운영 경로의 조립은 ConsoleAuditLogger 한 곳. 그러나 미사용 레거시 클래스 `FileAuditLogger`(AuditLogger 미구현, 참조 없음, git 미추적)가 `"[AUDIT] " + line` 접두를 별도로 조립한다. `ApprovalAuditRecorder:27` 의 `"type=" + 유형` 은 상세 내용 생성이라 형식 조립에 해당하지 않음.

3.2 판정 요약

충족 **7** / 부분 충족 **1** / 미충족 **0** / 불일치 **0** / 확인 불가 **0**

4. 구성 요소 설계

클래스	책임	주요 메서드 시그니처 (현행 유지)
`common.AuditLogger` (interface)	감사로그 추상화	`void write(String action, Long id, Long userId)` `void write(String action, Long id, Long userId, String detail)`
`common.ConsoleAuditLogger` (@Component)	형식 조립·콘솔 출력 (유일한 조립 지점)	`public ConsoleAuditLogger()`, `public ConsoleAuditLogger(Clock clock)`, `private String format(...)`, `private void print(String line)`
`approval.ApprovalAuditRecorder` (@Component)	결재 동작 → 동작 이름 매핑, 생성 시 `type=` 상세	`recordCreated / recordSubmitted / recordApproved / recordRejected / recordCanceled(Approval approval, Long actorId)`
`notice.NoticeService`	NOTICE CREATE / PUBLISH 기록	`create(...)`, `publish(Long id, Long userId)`
`schedule.ScheduleService`	SCHEDULE CREATE / CONFIRM 기록	`create(...)`, `confirm(Long id, Long userId)`
`common.ClockConfig`	시각 출처 Clock 빈 (REQ-NF-004)	-

5. 인터페이스 설계

파라미터	타입	설명	예시
action	String	동작 이름 (9종 고정)	APPROVAL SUBMIT
id	Long	대상 ID (결재·공지·일정)	3
userId	Long	수행자 ID	1
detail	String (선택)	끝에 공백 하나와 붙일 상세	type=1

출력 예시

```
[AUDIT] [2026-10-17 09:30:00] APPROVAL CREATE id=1 by=1 type=1
[AUDIT] [2026-10-17 09:30:00] APPROVAL SUBMIT id=1 by=1
```

```
[AUDIT] [2026-10-17 09:30:00] NOTICE PUBLISH id=3 by=2
[AUDIT] [2026-10-17 09:30:00] SCHEDULE CONFIRM id=5 by=4
```

동작 이름표

동작 이름	계기 (상태 변경)	상세	기록 위치
APPROVAL CREATE	결재 생성 → 0 임시	type={유형}	`ApprovalAuditRecorder.java:27`
APPROVAL SUBMIT	0 → 1	없음	`:31`
APPROVAL APPROVE	1 → 2	없음	`:35`
APPROVAL REJECT	1 → 3	없음	`:39`
APPROVAL CANCEL	→ 9 취소	없음	`:43`
NOTICE CREATE	공지 생성 → 0 임시	없음	`NoticeService.java:69`
NOTICE PUBLISH	0 → 1 게시	없음	`NoticeService.java:94`
SCHEDULE CREATE	일정 생성 → 0 예정	없음	`ScheduleService.java:54`
SCHEDULE CONFIRM	0 → 1 확정	없음	`ScheduleService.java:91`

오류 동작: 예외를 던지지 않는다. id·userId·detail 이 null 이면 문자열 연결 결과 `null` 로 찍힌다(보존).

6. 데이터 설계

감사로그는 DB 에 저장하지 않는다(콘솔 출력). 기록 계기가 되는 상태값은 DB 정수 그대로다: 결재 0/1/2/3/9, 공지 0/1/9, 일정 0/1/9. 시각은 주입 `Clock`(시스템 기본 시간대)의 현재 시각을 `yyyy-MM-dd HH:mm:ss` 로 표기.

7. 처리 흐름

순서	단계	처리	소스 (결재 상신 예)
1	가드	엔티티 행위(`submit`·`approve`·`publish`·`confirm` 등)가 false → return, 기록 없음	`ApprovalService.java:140-142`
2	상태 변경·저장	`repository.save`	`ApprovalService.java:143`
3	메일 (해당 시)	REQ-IF-001	`ApprovalService.java:144`
4	동작 이름 결정	`ApprovalAuditRecorder.recordSubmitted`	`ApprovalService.java:145`
5	형식 조립	`ConsoleAuditLogger.format` - 시각·동작·id·by (+상세)	`ConsoleAuditLogger.java:54-57`
6	출력	`[AUDIT]` 접두 후 println	`ConsoleAuditLogger.java:59-61`

8. 예외 · 경계 처리

상황	현재 동작	레거시 보존 여부
조건 불만족으로 무시된 처리 (결재·공지·일정)	기록 없음	보존
일정 등록 거부 (검증·겹침)	기록 없음	보존
대상 id / 수행자 id null	`id=null / by=null` 출력	보존 (소스 주석 명시, 테스트 미고정)
상세 null 로 4인자 호출	끝에 `null` 이 붙음	보존 (현재 호출부는 항상 "type=..." 전달)
Clock 빈 없는 슬라이스 테스트	기본 생성자 → 시스템 기본 Clock	보존

9. 검증 설계

검증 기준 (F열)	고정 테스트
조건 불만족으로 무시된 처리에는 감사로그가 없다.	`ApprovalNotificationOutputTest#무시되는_처리는_메일도_감사로그도_남기지_않는다` (보조) `ScheduleServiceCharacterizationTest#소유자가_아닌_사용자가_확정하면_조용히_무시된다`, `NoticeServiceCharacterizationTest#이미_게시된_공지를_다시_게시하면_무시되고_로그도_한번만_남는다`, `ApprovalServiceMailUnitTest#권한없는_사원이_승인하면_메일도_감사로그도_추가되지_않는다`
결재 생성 로그 끝에 'type={유형}' 이 붙는다.	`ApprovalNotificationOutputTest#결재를_생성하면_유형이_붙은_감사로그가_남는다` (보조) `ClockInjectionTest#고정_Clock이면_감사로그_시각이_고정_시각이다`

추가 테스트 제안

제안 메서드명	내용 (변수는 영어)
`고정_Clock이면_감사로그_한줄이_정확히_형식과_같다(CapturedOutput output)`	`Clock fixedClock 으로 new ConsoleAuditLogger(fixedClock).write("NOTICE PUBLISH", 3L, 2L) → 출력이 정확히 "[AUDIT] [2026-10-17 09:30:00] NOTICE PUBLISH id=3 by=2" + lineSeparator`
`상세가_있으면_공백_하나로_끝에_붙는다(CapturedOutput output)`	`write("APPROVAL CREATE", 1L, 1L, "type=2") → 끝이 " by=1 type=2"`
`대상_id가_null이면_null로_찍힌다(CapturedOutput output)`	`write("SCHEDULE CREATE", null, 4L) → "id=null by=4"` (현재 동작 고정)

10. 검토 의견 · 후속 과제

구분	내용
보존 대상	`AuditLogger` 두 오버로드 시그니처, 줄 형식·시각 패턴·`[AUDIT]` 접두, 동작 이름 9종, 결재 생성만 `type=` 상세, 상태 변경 시에만 기록, 저장 → 메일 → 감사로그 순서, null 값 문자열 출력.
개선 대상	(1) [부분 충족 설계안] 미사용 레거시 `common/FileAuditLogger.java` 를 제거하거나, 파일 기록이 필요하면 `AuditLogger` 를 구현하고 형식은 공통 포매터(예: `AuditLineFormat` 정적 메서드)를 ConsoleAuditLogger 와 공유하도록 바꾼다 - 형식 조립 지점을 하나로. 운영 동작 영향 없음, 파일 삭제는 사용자 확인 후. (2) 동작 이름 문자열 → `AuditAction` enum(출력 문자열 동일). (3) `System.out` → 로거/영속 저장(감사 요건 시). (4) 생성 로그 상세 `type=` 외 확장 규칙 미정.

구분	내용
연계 요구 사항	REQ-F-001~005(결재), REQ-F-009 · REQ-F-010(공지), REQ-F-011 · REQ-F-012(일정), REQ-IF-001(메일 다음 기록), REQ-NF-004(Clock 주입)